บทที่ 9: การประมวลผลสตรีม (**Stream Processing)**

(*สรุปเป็นภาษาไทยจากฉบับที่* ***1*** *ใช้เวลาฟังประมาณ* ***18–20*** *นาที)*

ตอนที่ 1: บทนำ

สวัสดีครับ ยินดีต้อนรับสู่บทสรุปหนังสือ **Designing Data-Intensive Applications** บทที่เก้า ว่าด้วยการประมวลผลสตรีม หรือ **Stream Processing**

ในสามบทที่ผ่านมา เราพูดถึงข้อมูลแบบนิ่ง ๆ ที่นอนรออยู่ในฐานข้อมูลหรือคลังข้อมูล แต่ในโลกความเป็นจริง ข้อมูลส่วนใหญ่ไหลมาเรื่อย ๆ ไม่มีวันสิ้นสุด เช่น การคลิกหน้าเว็บของลูกค้า การรูดบัตรเครดิต หรือข้อมูลจากเซนเซอร์ **IoT** ข้อมูลที่ไหลมาแบบไม่หยุดนี้เรียกว่า สตรีม (**Stream)**

ในอดีต ระบบมักรอให้ข้อมูลสะสมเป็นก้อนใหญ่แล้วค่อยประมวลผลเป็นรอบ ๆ เรียกว่า **Batch Processing** ซึ่งช้าเกินไปสำหรับโลกยุคใหม่ที่ต้องการผลลัพธ์แบบเรียลไทม์

บทนี้จะพาไปทำความรู้จักกับการประมวลผลสตรีม ว่าข้อมูลถูกส่งต่ออย่างไร ถูกประมวลผลแบบไหน และเราจะรับมือกับปัญหาเรื่อง "เวลา" ที่ยุ่งเหยิงในระบบกระจายได้อย่างไรครับ

ตอนที่ 2: การส่งต่อสตรีม (**Transmitting Event Streams)**

สตรีมคือข้อมูลที่ถูกสร้างขึ้นครั้งเดียวแล้วส่งต่อไปเรื่อย ๆ ฝั่งที่สร้างข้อมูลเรียกว่า **Producers (**ผู้ผลิต) ส่วนฝั่งที่รับไปใช้เรียกว่า **Consumers (**ผู้บริโภค)

ในระบบแบบดั้งเดิม เรามักใช้ฐานข้อมูลคอยเช็กว่ามีข้อมูลใหม่ไหม แต่การคอยวนลอบถามฐานข้อมูลตลอดเวลา (**Polling)** นั้นเปลืองทรัพยากรมาก ระบบสตรีมยุคใหม่จึงเปลี่ยนมาใช้แนวคิดการส่งข้อความผ่านตัวกลาง หรือ **Message Broker** ซึ่งมีสองรูปแบบหลัก

รูปแบบแรก: **Messaging Queues (**คิวข้อความ) เช่น **RabbitMQ** หรือ **Celery** วิธีนี้เหมาะกับการกระจายงานหนักให้คนช่วยกันทำ เมื่อมีข้อความเข้ามา ระบบจะส่งให้ผู้บริโภคไปทำทีละชิ้น เมื่อทำเสร็จข้อความนั้นก็จะหายไป ผู้บริโภคแต่ละตัวจะได้รับงานไม่ซ้ำกัน เหมาะกับงานเบื้องหลังที่ต้องการความรวดเร็ว

รูปแบบที่สอง: **Event Logs (**บันทึกเหตุการณ์) เช่น **Apache Kafka** หรือ **AWS Kinesis** วิธีนี้ต่างออกไป ข้อมูลที่เข้ามาจะไม่หายไปไหน แต่จะถูกเขียนต่อท้ายเรียงยาวเป็นบันทึก (**Log)** เหมือนประวัติศาสตร์ที่แก้ไขไม่ได้ ผู้บริโภคแต่ละรายสามารถมาอ่านข้อมูลชุดเดียวกันซ้ำได้ตามต้องการ โดยต่างคนต่างจำตำแหน่ง (**Offset)** ของตัวเองว่าอ่านไปถึงไหนแล้ว วิธีนี้ได้รับความนิยมสูงมากในระบบข้อมูลขนาดใหญ่ เพราะปลอดภัยและย้อนกลับมาดูประวัติเก่าได้เสมอ

ตอนที่ 3: ฐานข้อมูลกับสตรีมมีความเกี่ยวข้องกันอย่างไร

ผู้เขียนชี้ให้เห็นมุมมองที่น่าสนใจมากว่า ฐานข้อมูลกับสตรีมมีความเกี่ยวโยงกันอย่างลึกซึ้ง

จริง ๆ แล้ว ฐานข้อมูลก็คือสตรีมที่ถูกเก็บบันทึกไว้นิ่ง ๆ (**Log** ของฐานข้อมูล) และสตรีมก็คือฐานข้อมูลที่กำลังเดินทางผ่านกาลเวลา

แนวคิดนี้ทำให้เกิดสถาปัตยกรรมที่เรียกว่า **Change Data Capture (CDC)** คือถ้าฐานข้อมูลมีการเปลี่ยนแปลงอะไรเกิดขึ้น (เพิ่ม ลบ แก้ไข) ระบบจะส่งการเปลี่ยนแปลงนั้นออกเป็นสตรีมทันที เพื่อให้ระบบอื่น เช่น คลังข้อมูลหรือระบบค้นหา ได้อัปเดตตามแบบเรียลไทม์โดยไม่ต้องคอยไปดึงข้อมูลก้อนใหญ่มาตรวจเช็กเอง

นอกจากนี้ยังมีแนวคิด **Event Sourcing** คือการเก็บทุกการกระทำที่เกิดขึ้นกับระบบเป็นสตรีมเหตุการณ์ดิบ ๆ แทนที่จะเก็บแค่สถานะปัจจุบัน เช่น แทนที่จะเก็บยอดเงินในบัญชีปัจจุบัน **2,000** บาท ระบบจะเก็บบันทึกสตรีมว่า "ฝาก **1,000"** แล้ว "ฝากอีก **1,000"** อยากรู้ยอดตอนไหนก็เอาสตรีมมาวิ่งคำนวนเอาทีหลัง ทำให้ตรวจสอบย้อนหลังและแกะรอยข้อผิดพลาดได้ง่ายมาก

ตอนที่ **4:** การประมวลผลสตรีมทำอะไรได้บ้าง

เมื่อเรามีสตรีมข้อมูลไหลผ่านระบบแล้ว เราเอาไปทำอะไรได้บ้าง**?**

1. เขียนลงฐานข้อมูล (**Write to Storage):** แปลงข้อมูลสตรีมแล้วบันทึกลงฐานข้อมูลหรือคลังข้อมูลเพื่อให้อ่านง่าย
2. แจ้งเตือน (**Waking up User / Sending Notifications):** เฝ้าระวังเหตุการณ์ เช่น ตรวจจับการรูดบัตรเครดิตที่ผิดปกติ แล้วส่งแจ้งเตือนเข้ามือถือผู้ใช้ทันที
3. ประมวลผลเชิงซ้อน (**Stream Analytics):** คำนวณสถิติต่าง ๆ แบบเรียลไทม์ เช่น นับจำนวนคำขอต่อนาที หรือคำนวณยอดขายเฉลี่ยย้อนหลัง **5** นาที

เครื่องมือที่ใช้ประมวลผลสตรีมยอดนิยมในปัจจุบัน เช่น **Apache Flink, Apache Storm** และ **Kafka Streams**

ตอนที่ **5:** ปัญหาเรื่อง "เวลา" ในสตรีม

หัวข้อนี้คือหัวใจสำคัญที่สุดและปวดหัวที่สุดของการประมวลผลสตรีมครับ

เวลาเราคำนวณสตรีม เช่น "ขอทราบยอดขายรวมในแต่ละชั่วโมง" คำถามคือ เราจะรู้ได้อย่างไรว่าครบหนึ่งชั่วโมงแล้ว**?**

ในระบบกระจาย ข้อมูลไม่ได้เดินทางมาถึงเรียงตามเวลาที่มันเกิดขึ้นจริงเสมอไป เช่น ลูกค้าซื้อของตอน **12:59** น. แต่เพราะสัญญาณอินเทอร์เน็ตหลุด ข้อมูลเพิ่งเดินทางมาถึงระบบตอน **13:15** น.

ถ้าเราใช้นาฬิกาของเครื่องประมวลผล (**Processing Time)** เป็นตัวตัดรอบ ข้อมูลชิ้นนั้นจะถูกนัดไปคิดเป็นยอดของ **13:15** น. ซึ่งผิดความจริงไปไกลมาก

ระบบจึงต้องแยกเวลาออกเป็นสองแบบ:

- **Event Time:** เวลาที่เหตุการณ์นั้นเกิดขึ้นจริง ๆ บนเครื่องของลูกค้า
- **Processing Time:** เวลาที่ระบบคอมพิวเตอร์ของเราได้รับข้อมูลนั้น

การใช้ **Event Time** ถูกต้องที่สุด แต่ปัญหาก็คือ เราไม่มีทางรู้เลยว่าข้อมูลชิ้นสุดท้ายของชั่วโมงนั้นจะเดินทางมาถึงเมื่อไร ถ้ามีข้อมูลตกค้างเดินทางมาช้ามาก ๆ เราควรจะรอนานแค่ไหน หรือจะตัดรอบแค่นั้นแล้วทิ้งข้อมูลที่มา**ou**ช้าดี**?**

ระบบสตรีมยุคใหม่จึงต้องมีเครื่องมือจัดการเรื่องนี้ เช่น การใช้ **Watermarks (**เส้นแบ่งเวลาสมมติ) เพื่อบอกระบบว่า "เรามั่นใจว่าข้อมูลจนถึงเวลานี้เดินทางมาครบหมดแล้วนะ เริ่มคำนวณผลลัพธ์รอบนี้ได้" และถ้ามีข้อมูลหลงยุค (**Stragglers)** เดินทางมาทีหลัง ระบบก็ต้องมีวิธีจัดการ เช่น คำนวณแก้ไขยอดใหม่ หรือส่งรายงานเฉพาะส่วนที่ตกหล่นไปปรับปรุง

ตอนที่ **6:** การเข้าร่วมสตรีม (**Stream Joins)**

อีกหนึ่งความท้าทายคือการเอาสตรีมสองสายมาเชื่อมกัน (**Join)** ซึ่งต่างจากการ **Join** ในฐานข้อมูลปกติที่มีข้อมูลนิ่ง ๆ รออยู่

การ **Join** ในสตรีมมี **3** แบบหลัก:

1. **Stream-Table Join:** เอาสตรีมเหตุการณ์มาจับคู่กับข้อมูลนิ่งในฐานข้อมูล เช่น มีสตรีมการคลิกสินค้าวิ่งเข้ามา อยากรู้ว่าสินค้านั้นราคาเท่าไร ก็ต้องวิ่งไปดึงข้อมูลราคาน่าปัจจุบันจากฐานข้อมูลมาแปะร่วม
2. **Stream-Stream Join (Window Join):** เอาสตรีมสองสายมาจับคู่กันในช่วงเวลาเดียวกัน เช่น ลูกค้าค้นหาคำว่า "รองเท้า" บนเว็บ (สตรีมที่หนึ่ง) แล้วต้องคลิกซื้อสินค้าภายใน **1** นาทีต่อมา (สตรีมที่สอง) ระบบต้องจำเหตุการณ์แรกไว้ชั่วคราวเพื่อรอจับคู่กับเหตุการณ์ที่สอง
3. **Stream-Stream Join (Stateful Join):** การเชื่อมสตรีมสองสายที่ไม่มีกรอบเวลาจำกัด แต่ต้องเก็บสถานะสะสมไว้เรื่อย ๆ เช่น การจับคู่โฆษณากับยอดคลิกซื้อที่อาจเกิดขึ้นข้ามวัน

ตอนที่ **7:** ความถูกต้องและความทนทาน (**Fault Tolerance)**

ในการประมวลผลสตรีม คำถามที่ต้องตอบให้ได้คือ ถ้าเครื่องพังกลางทาง ข้อมูลจะถูกนับซ้ำสองรอบไหม**?**

ในโลก **Batch Processing** เราสั่งรันใหม่ตั้งแต่ต้นได้ง่าย แต่ในสตรีมที่ข้อมูลไหลมาเรื่อย ๆ ไม่มีวันจบ การรันใหม่ไม่ง่ายแบบนั้น

เป้าหมายสูงสุดคือ **Exactly-once semantics (**ประมวลผลถูกต้องครบถ้วนพอดีเป๊ะ แม้เครื่องจะพังกลางคัน เหมือนไม่มีอะไรเกิดขึ้น)

ระบบสตรีมสมัยใหม่ (เช่น **Apache Flink** ร่วมกับ **Kafka)** บรรลุเป้าหมายนี้ได้ด้วยเทคนิคการบันทึกภาพสถานะเป็นระยะ (**Checkpointing)** และการออกแบบให้การบันทึกผลลัพธ์ลงปลายทางเป็นแบบ **Idempotent (**บันทึกซ้ำกี่รอบ ผลลัพธ์ก็ไม่เพี้ยน) หรือใช้ธุรกรรมขนาดเล็กครอบคลุมการอ่าน-ประมวลผล-เขียนไว้ด้วยกัน

ตอนที่ **8:** สรุปท้ายบท

มาทบทวนประเด็นสำคัญของบทนี้กันครับ

หนึ่ง สตรีมคือข้อมูลที่ไหลมาไม่หยุด ระบบจัดเก็บแบบ **Event Log (เช่น Kafka)** ช่วยให้เราเก็บและส่งต่อสตรีมได้อย่างปลอดภัยและย้อนกลับมาดูได้

สอง ฐานข้อมูลกับสตรีมมีความเกี่ยวโยงกัน แนวคิดอย่าง **CDC** และ **Event Sourcing** ช่วยเปลี่ยนมุมมองการสร้างระบบให้เป็นแบบเรียลไทม์มากขึ้น

สาม การจัดการกับ "เวลา" (**Event Time** เทียบกับ **Processing Time)** และการใช้ **Watermarks** คือหัวใจสำคัญในการคำนวณสตรีมให้ถูกต้องแม้เครือข่ายจะหน่วง

สี่ การประมวลผลสตรีมต้องรับมือกับการเชื่อมข้อมูล (**Joins)** และความถูกต้องเมื่อเครื่องพัง (**Fault Tolerance / Exactly-once)** เพื่อให้มั่นใจว่าตัวเลขที่คำนวณออกมาเชื่อถือได้จริง

นี่คือบทสุดท้ายของเนื้อหาหลักในหนังสือเล่มนี้แล้วครับ เป็นการเดินทางที่ยาวนานตั้งแต่เครื่องเดี่ยว ระบบกระจาย จนถึงสตรีมมิ่งแบบเรียลไทม์

ต้องการให้สรุปภาพรวมทั้งหมดของหนังสือ หรือเจาะลึกบทไหนเป็นพิเศษไหมครับ 😊